AI & MACHINE LEARNING WITH PYTHON

The Complete Practitioner's Handbook

Covers: Supervised & Unsupervised Learning · Deep Learning · NLP · Computer Vision
Reinforcement Learning · MLOps · Model Deployment · Ethics & Best Practices

scikit-learn · TensorFlow · PyTorch · Keras · Hugging Face · OpenCV

Ashish Kumar
Senior Technical Leader, KPIT Technologies Limited

Bachelor of Technology in Electrical and Electronics Engineering, Gaya College of Engineering
Master of Technology in Power System Engineering, Indian Institute of Technology (ISM) Dhanbad
Post Graduate Certification in General Management, Indian Institute of Management Nagpur

This comprehensive handbook has been designed using Claude AI (Sonnet 4.6) and prompts as provided to the GPT.

© 2025 AI & ML Handbook

Preface

This handbook is a comprehensive guide for anyone seeking to understand, implement, and deploy Artificial Intelligence and Machine Learning systems using Python. Whether you are a software engineer moving into ML, a data scientist deepening your expertise, or a researcher looking for practical implementation details, this handbook covers the full breadth of the field.

Python has emerged as the definitive language for AI and ML work, owing to its expressive syntax, rich ecosystem of libraries, and strong community. This handbook leverages that ecosystem—from NumPy and Pandas for data manipulation to PyTorch and TensorFlow for deep learning—and teaches you how these tools fit together in real projects.

How to Use This Handbook

Each chapter builds on the previous, but experienced practitioners may jump directly to topics of interest. Code examples are self-contained and runnable. Mathematical notation is introduced gradually and always accompanied by intuitive explanations.

- Chapters 1–3: Foundations — Python stack, data wrangling, and statistical thinking
- Chapters 4–6: Classical ML — supervised, unsupervised, and ensemble methods
- Chapters 7–9: Deep Learning — neural networks, CNNs, and RNNs
- Chapters 10–12: NLP & Computer Vision — modern transformers and vision models
- Chapters 13–14: Reinforcement Learning and Generative AI
- Chapter 15: MLOps — deployment, monitoring, and production systems
- Chapter 16: Ethics, fairness, and responsible AI

CHAPTER 1

Foundations

The Python AI Ecosystem

1.1 The AI and ML Landscape

Artificial Intelligence is the broad field concerned with creating machines that can perform tasks that typically require human intelligence. Machine Learning is a subfield of AI in which systems learn from data rather than being explicitly programmed. Deep Learning is a further subset that uses multilayer neural networks to automatically discover hierarchical representations in data.

Key Distinction

Traditional programming: Rules + Data → Output. Machine Learning: Data + Output → Rules (model). This inversion is fundamental to understanding why ML is transformative.

1.2 Python Environment Setup

A reproducible Python environment is the foundation of any ML project. Use virtual environments or Conda to isolate dependencies.

1.2.1 Creating a Virtual Environment

```
python -m venv mlenv
source mlenv/bin/activate        # macOS/Linux
mlenv\Scripts\activate.bat      # Windows
pip install --upgrade pip
```

1.2.2 Core Libraries

```
pip install numpy pandas matplotlib seaborn scikit-learn
pip install torch torchvision torchaudio  # PyTorch
pip install tensorflow keras              # TensorFlow / Keras
pip install transformers datasets         # Hugging Face
pip install xgboost lightgbm catboost
pip install opencv-python pillow          # Computer Vision
pip install jupyterlab notebook ipywidgets
```

1.3 NumPy — Numerical Computing

NumPy underpins virtually all Python ML libraries. Its ndarray provides fast, vectorized operations on homogeneous data.

1.3.1 Array Creation and Manipulation

```
import numpy as np

# Creation
a = np.array([1, 2, 3, 4, 5])           # 1-D array
b = np.zeros((3, 4))                     # 3x4 matrix of zeros
c = np.ones((2, 3, 4))                   # 3-D tensor of ones
```

© 2025 AI & ML Handbook

```
d = np.arange(0, 10, 0.5)                # range with step
e = np.linspace(0, 1, 100)               # 100 evenly spaced points
f = np.random.randn(1000, 784)           # standard normal

# Shape operations
a.reshape(5, 1)                           # (5,) → (5,1) column
np.vstack([b, b])                         # vertical stack → (6,4)
np.hstack([b, b])                         # horizontal stack → (3,8)
b.T                                       # transpose

# Vectorised math (no Python loops)
x = np.array([1.0, 2.0, 3.0])
sigmoid = 1 / (1 + np.exp(-x))           # element-wise sigmoid
dot_product = x @ x                      # dot product = 14.0
```

1.3.2 Broadcasting

Broadcasting allows NumPy to perform operations on arrays of different shapes without copying data.

```
A = np.ones((3, 4))
v = np.array([1, 2, 3, 4])               # shape (4,)
result = A + v                           # broadcasts to (3,4)
# Each row of A has v added to it
```

1.4 Pandas — Data Manipulation

Pandas provides DataFrame and Series objects for working with structured (tabular) data. It is the workhorse for data loading, cleaning, and feature engineering.

1.4.1 Loading and Inspecting Data

```
import pandas as pd

df = pd.read_csv("data.csv")
df.head()                                 # first 5 rows
df.info()                                 # dtypes and non-null counts
df.describe()                             # summary statistics
df.shape                                  # (rows, cols)
df.dtypes                                 # column types
df.isnull().sum()                         # missing values per column
```

1.4.2 Selection and Filtering

```
# Column selection
df["age"]                                  # Series
df[["age", "income"]]                     # DataFrame

# Row filtering
df[df["age"] > 30]
df.query("age > 30 and income > 50000")
df.loc[df["city"] == "NYC", ["age", "income"]]

# iloc: positional indexing
df.iloc[0:5, 2:5]                         # rows 0-4, cols 2-4
```

1.4.3 Data Cleaning

```
# Handle missing values
df.dropna()                              # drop rows with any NaN
df.fillna(df.mean(numeric_only=True))   # fill with column mean
df["age"].fillna(df["age"].median(), inplace=True)

# Remove duplicates
df.drop_duplicates(subset=["user_id"], keep="first", inplace=True)

# Type conversion
df["date"] = pd.to_datetime(df["date"])
df["category"] = df["category"].astype("category")
```

1.4.4 GroupBy and Aggregation

```
# Group by and aggregate
df.groupby("city")["income"].agg(["mean", "median", "std"])

pivot = df.pivot_table(
    values="sales", index="region",
    columns="quarter", aggfunc="sum", fill_value=0
)
```

1.5 Matplotlib & Seaborn — Visualization

1.5.1 Matplotlib Essentials

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Line plot
axes[0].plot(x, y, color="steelblue", linewidth=2, label="Train")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss")
axes[0].set_title("Training Curve"); axes[0].legend()

# Scatter
axes[1].scatter(X[:,0], X[:,1], c=y, cmap="coolwarm", alpha=0.7)
plt.tight_layout(); plt.savefig("plot.png", dpi=150)
```

1.5.2 Seaborn Statistical Plots

```
import seaborn as sns

sns.histplot(df["income"], bins=40, kde=True)
sns.boxplot(data=df, x="region", y="sales")
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
sns.pairplot(df, hue="target", diag_kind="kde")
```

© 2025 AI & ML Handbook

CHAPTER 2

Data Preparation

Feature Engineering & Preprocessing

2.1 The Data Preprocessing Pipeline

Raw data is almost never fit for direct use in ML models. A systematic preprocessing pipeline ensures consistency between training and inference and prevents data leakage.

> **Data Leakage Warning**
> Always fit preprocessing transformers on training data only, then apply (transform) to both training and test sets. Fitting on the full dataset leaks information from the test set into the model.

2.2 Handling Missing Values

2.2.1 Imputation Strategies

The right strategy depends on the missingness mechanism (MCAR, MAR, MNAR) and the feature type.

Strategy	Use When	Code
Mean/Median	Numeric, few missing	`SimpleImputer(strategy="median")`
Mode	Categorical	`SimpleImputer(strategy="most_frequent")`
KNN	Small datasets	`KNNImputer(n_neighbors=5)`
Iterative	Complex patterns	`IterativeImputer(estimator=BayesianRidge())`
Indicator	Missingness is informative	`MissingIndicator() + imputer`

2.3 Feature Scaling

Most ML algorithms are sensitive to the scale of input features. Standardization or normalization is critical for distance-based, gradient-based, and regularized models.

```
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, RobustScaler, PowerTransformer
)

# StandardScaler: mean=0, std=1 — best for most algorithms
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)   # NO fit here!

# MinMaxScaler: [0,1] — use when distribution is bounded
mm = MinMaxScaler(feature_range=(0, 1))
```

```
# RobustScaler: median/IQR — robust to outliers
rb = RobustScaler(quantile_range=(25, 75))

# PowerTransformer: Yeo-Johnson — normalise skewed distributions
pt = PowerTransformer(method="yeo-johnson", standardize=True)
```

2.4 Encoding Categorical Variables

2.4.1 Ordinal Encoding

```
from sklearn.preprocessing import OrdinalEncoder
enc = OrdinalEncoder(categories=[["low","medium","high"]])
X["grade_encoded"] = enc.fit_transform(X[["grade"]])
```

2.4.2 One-Hot Encoding

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore")
ohe.fit_transform(X[["city", "product_type"]])
```

2.4.3 Target Encoding

```
# High-cardinality features: encode with target mean (+ smoothing)
from category_encoders import TargetEncoder
enc = TargetEncoder(smoothing=10)
X_enc = enc.fit_transform(X[["zip_code"]], y)
```

2.4.4 Hashing Trick

```
from sklearn.feature_extraction import FeatureHasher
fh = FeatureHasher(n_features=2**12, input_type="string")
X_hashed = fh.transform(X["category"].apply(lambda x: [x]))
```

2.5 Feature Engineering

Feature engineering — creating new features from raw data — is often the most impactful step in an ML pipeline.

2.5.1 Datetime Features

```
df["hour"]          = df["timestamp"].dt.hour
df["day_of_week"]   = df["timestamp"].dt.dayofweek
df["month"]         = df["timestamp"].dt.month
df["is_weekend"]    = df["day_of_week"].isin([5, 6]).astype(int)
df["days_since"]    = (pd.Timestamp.now() - df["timestamp"]).dt.days
```

2.5.2 Interaction Features

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_poly = poly.fit_transform(X_numeric)
```

2.5.3 Binning

```
# Equal-width bins
df["age_bin"] = pd.cut(df["age"], bins=5, labels=["<20","20-35","35-50","50-65","65+"])
```

© 2025 AI & ML Handbook

```
# Quantile bins (equal frequency)
df["income_qtile"] = pd.qcut(df["income"], q=4, labels=["Q1","Q2","Q3","Q4"])
```

2.6 Feature Selection

2.6.1 Filter Methods

```
from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif
sel = SelectKBest(score_func=mutual_info_classif, k=20)
X_sel = sel.fit_transform(X_train, y_train)
selected = X.columns[sel.get_support()]
```

2.6.2 Embedded Methods (L1 Regularization)

```
from sklearn.linear_model import LassoCV
from sklearn.feature_selection import SelectFromModel
lasso = LassoCV(cv=5, random_state=42).fit(X_train, y_train)
sel = SelectFromModel(lasso, prefit=True)
X_lasso = sel.transform(X_train)
```

2.6.3 Recursive Feature Elimination

```
from sklearn.feature_selection import RFECV
from sklearn.ensemble import GradientBoostingClassifier
rfecv = RFECV(estimator=GradientBoostingClassifier(), cv=5, scoring="roc_auc")
rfecv.fit(X_train, y_train)
print(f"Optimal features: {rfecv.n_features_}")
```

2.7 Scikit-learn Pipelines

Pipelines chain transformers and an estimator into a single object. This prevents data leakage, simplifies cross-validation, and enables seamless deployment.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier


numeric_feats  = ["age", "income", "balance"]
categoric_feats = ["city", "product"]

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler",  StandardScaler()),
])

categoric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("ohe",     OneHotEncoder(drop="first", handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
```

```
    ("num", numeric_pipe,  numeric_feats),
    ("cat", categoric_pipe, categoric_feats),
])

full_pipeline = Pipeline([
    ("prep",  preprocessor),
    ("model", RandomForestClassifier(n_estimators=300, random_state=42)),
])

full_pipeline.fit(X_train, y_train)
preds = full_pipeline.predict(X_test)
```

© 2025 AI & ML Handbook

CHAPTER 3

Supervised Learning

Regression & Classification

3.1 The Supervised Learning Framework

In supervised learning, we have a dataset of (input, label) pairs and seek a function f such that f(x) ≈ y. The model is trained by minimizing a loss function that measures the discrepancy between predictions and true labels.

Bias–Variance Tradeoff

A model's test error decomposes as: Error = Bias² + Variance + Irreducible Noise. High bias → underfitting (model too simple). High variance → overfitting (model memorizes training data). Regularization, cross-validation, and ensemble methods manage this tradeoff.

3.2 Linear Regression

Linear regression models the target as a linear combination of features: $\hat{y} = w_0 + w_1x_1 + \ldots + w_nx_n$. Ordinary Least Squares minimizes the sum of squared residuals.

3.2.1 Ordinary Least Squares

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score


lr = LinearRegression(fit_intercept=True)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)


rmse = mean_squared_error(y_test, y_pred, squared=False)
r2   = r2_score(y_test, y_pred)
print(f"RMSE={rmse:.3f}  R²={r2:.3f}")
print(f"Coefficients: {lr.coef_}")
```

3.2.2 Regularized Regression

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet, RidgeCV


# Ridge (L2): shrinks coefficients, keeps all features
ridge = Ridge(alpha=1.0)


# Lasso (L1): shrinks some coefficients to zero (feature selection)
lasso = Lasso(alpha=0.01, max_iter=10000)


# ElasticNet: L1 + L2 combination
enet = ElasticNet(alpha=0.01, l1_ratio=0.5)


# Automatic alpha selection via cross-validation
ridge_cv = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
```

```
ridge_cv.fit(X_train, y_train)
print(f"Best alpha: {ridge_cv.alpha_}")
```

3.3 Logistic Regression

Logistic regression applies the sigmoid function to a linear combination of features to model binary classification probabilities: $P(y=1|x) = \sigma(w \cdot x + b)$.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix


clf = LogisticRegression(
    penalty="l2", C=1.0,                # C = 1/regularization strength
    solver="lbfgs", max_iter=1000,
    class_weight="balanced",            # handles imbalanced classes
    multi_class="auto"                  # auto-selects OvR or multinomial
)
clf.fit(X_train, y_train)


y_prob = clf.predict_proba(X_test)[:, 1]
y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
```

3.4 Decision Trees

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree


dt = DecisionTreeClassifier(
    criterion="gini",           # or "entropy" (information gain)
    max_depth=5,                # limit depth to prevent overfitting
    min_samples_leaf=20,        # minimum samples per leaf
    min_impurity_decrease=0.01, # only split if gain >= threshold
    class_weight="balanced",
    random_state=42
)
dt.fit(X_train, y_train)


# Visualise
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(20, 8))
plot_tree(dt, feature_names=feature_names, class_names=class_names,
          filled=True, rounded=True, ax=ax)
```

3.5 Support Vector Machines

SVMs find the maximum-margin hyperplane that separates classes. The kernel trick implicitly maps data to high-dimensional spaces, enabling non-linear classification.

```
from sklearn.svm import SVC, SVR
from sklearn.pipeline import make_pipeline
```

© 2025 AI & ML Handbook

```
# Classification
svm = make_pipeline(StandardScaler(), SVC(
    kernel="rbf",                # radial basis function
    C=10.0,                      # penalty for misclassification
    gamma="scale",              # kernel coefficient
    probability=True,           # enable predict_proba
    class_weight="balanced"
))
svm.fit(X_train, y_train)

# Regression
svr = make_pipeline(StandardScaler(), SVR(
    kernel="rbf", C=100, epsilon=0.1, gamma=0.01
))
```

3.6 K-Nearest Neighbors

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

knn = KNeighborsClassifier(
    n_neighbors=7,
    weights="distance",    # nearer neighbors have more influence
    metric="minkowski", p=2,   # Euclidean distance
    algorithm="ball_tree",     # efficient for high dimensions
    n_jobs=-1
)
knn.fit(X_train_scaled, y_train)
```

3.7 Model Evaluation

3.7.1 Cross-Validation

```
from sklearn.model_selection import cross_val_score, StratifiedKFold, KFold

# Stratified k-fold preserves class proportions
skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_val_score(clf, X, y, cv=skf, scoring="roc_auc", n_jobs=-1)
print(f"AUC = {scores.mean():.4f} ± {scores.std():.4f}")
```

3.7.2 Classification Metrics

Metric	Formula	Best For
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Balanced classes
Precision	TP/(TP+FP)	Costly false positives
Recall	TP/(TP+FN)	Costly false negatives
F1	2·P·R/(P+R)	Imbalanced classes
ROC-AUC	Area under ROC curve	Ranking models

PR-AUC	Area under PR curve	Severely imbalanced

3.7.3 Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import randint, uniform

# Grid search — exhaustive over discrete grid
param_grid = {
    "model__n_estimators": [100, 300, 500],
    "model__max_depth": [3, 5, 7, None],
    "model__min_samples_leaf": [1, 5, 10],
}
grid_search = GridSearchCV(full_pipeline, param_grid, cv=5,
    scoring="roc_auc", n_jobs=-1, verbose=1)
grid_search.fit(X_train, y_train)
print(grid_search.best_params_)

# Randomized search — efficient for large spaces
param_dist = {
    "model__n_estimators": randint(100, 1000),
    "model__max_depth": [3, 5, 7, 10, None],
    "model__learning_rate": uniform(0.01, 0.3),
}
rscv = RandomizedSearchCV(full_pipeline, param_dist, n_iter=50,
    cv=5, scoring="roc_auc", random_state=42, n_jobs=-1)
```

© 2025 AI & ML Handbook

CHAPTER 4

Ensemble Methods

Boosting, Bagging & Stacking

4.1 Random Forests

Random Forest trains an ensemble of decision trees on bootstrap samples of the data, with a random subset of features considered at each split. The final prediction aggregates (averages or majority votes) across all trees.

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor


rf = RandomForestClassifier(
    n_estimators=500,            # number of trees
    max_depth=None,              # grow fully unless limited
    max_features="sqrt",         # sqrt(n_features) per split for classification
    min_samples_leaf=4,
    bootstrap=True,              # sample with replacement
    oob_score=True,              # out-of-bag error estimate
    class_weight="balanced_subsample",
    n_jobs=-1, random_state=42
)
rf.fit(X_train, y_train)


# Feature importance
importances = pd.Series(rf.feature_importances_, index=feature_names)
importances.nlargest(20).plot(kind="barh")


# OOB accuracy (free validation set)
print(f"OOB Score: {rf.oob_score_:.4f}")
```

4.2 Gradient Boosting

Gradient boosting builds trees sequentially, with each tree fitting the residuals (negative gradient of the loss) of the ensemble so far. It is often the strongest out-of-the-box ML algorithm for tabular data.

4.2.1 Scikit-learn GradientBoosting

```
from sklearn.ensemble import GradientBoostingClassifier, HistGradientBoostingClassifier


# HistGradientBoosting: much faster, handles NaN natively
hgb = HistGradientBoostingClassifier(
    learning_rate=0.05,          # shrinkage — smaller = better
    max_iter=500,                # number of boosting stages
    max_leaf_nodes=31,           # controls tree complexity
    min_samples_leaf=20,
    l2_regularization=0.1,
    early_stopping=True,         # stops when val score plateaus
    validation_fraction=0.1,
    n_iter_no_change=20,
```

```
    random_state=42
)
```

4.2.2 XGBoost

```
import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=1000,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,               # row subsampling
    colsample_bytree=0.8,        # feature subsampling per tree
    reg_alpha=0.1,               # L1 regularization
    reg_lambda=1.0,              # L2 regularization
    min_child_weight=5,
    scale_pos_weight=neg/pos,   # imbalanced data
    use_label_encoder=False,
    eval_metric="auc",
    early_stopping_rounds=50,
    tree_method="hist",          # fast histogram-based
    device="cuda",               # GPU if available
    random_state=42
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=100)
```

4.2.3 LightGBM

```
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=2000,
    learning_rate=0.02,
    num_leaves=63,               # max 2^max_depth - 1
    min_child_samples=30,
    feature_fraction=0.8,
    bagging_fraction=0.8,
    bagging_freq=5,
    reg_alpha=0.3, reg_lambda=0.3,
    is_unbalance=True,
    n_jobs=-1, random_state=42
)
```

4.3 Stacking Ensembles

Stacking trains a meta-learner on the out-of-fold predictions of base learners. It combines diverse models to often outperform any individual model.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

base_learners = [
    ("rf",   RandomForestClassifier(n_estimators=300, random_state=42)),
    ("xgb",  xgb.XGBClassifier(n_estimators=300, random_state=42)),
    ("lgbm", lgb.LGBMClassifier(n_estimators=300, random_state=42)),
```

© 2025 AI & ML Handbook

```
    ("svm",  make_pipeline(StandardScaler(), SVC(probability=True))),
]

stack = StackingClassifier(
    estimators=base_learners,
    final_estimator=LogisticRegression(C=0.1),
    cv=5,
    stack_method="predict_proba",
    passthrough=True,   # also pass original features to meta-learner
    n_jobs=-1
)
```

CHAPTER 5

Unsupervised Learning

Clustering, Dimensionality Reduction & Anomaly Detection

5.1 Clustering

5.1.1 K-Means

```
from sklearn.cluster import KMeans, MiniBatchKMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Find optimal k with elbow method
inertias = []
sil_scores = []
K_range = range(2, 15)

for k in K_range:
    km = KMeans(n_clusters=k, init="k-means++", n_init=20, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
    sil_scores.append(silhouette_score(X_scaled, km.labels_, sample_size=5000))

# MiniBatchKMeans for large datasets
mbkm = MiniBatchKMeans(n_clusters=8, batch_size=10000, n_init=10)
```

5.1.2 DBSCAN — Density-Based Clustering

```
from sklearn.cluster import DBSCAN

# eps: neighborhood radius; min_samples: core point threshold
dbscan = DBSCAN(eps=0.5, min_samples=10, metric="euclidean", n_jobs=-1)
labels = dbscan.fit_predict(X_scaled)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise    = list(labels).count(-1)
print(f"Clusters: {n_clusters}, Noise points: {n_noise}")
```

5.1.3 Hierarchical Clustering

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

agg = AgglomerativeClustering(
    n_clusters=5, linkage="ward", metric="euclidean"
)
labels = agg.fit_predict(X_scaled)

# Dendrogram
Z = linkage(X_scaled, method="ward")
fig, ax = plt.subplots(figsize=(14, 6))
dendrogram(Z, truncate_mode="lastp", p=30, ax=ax)
```

© 2025 AI & ML Handbook

5.2 Dimensionality Reduction

5.2.1 Principal Component Analysis (PCA)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)          # keep 95% of variance
X_pca = pca.fit_transform(X_scaled)
print(f"Components: {pca.n_components_}")
print(f"Explained variance: {pca.explained_variance_ratio_.cumsum()[-1]:.3f}")

# Biplot of first two PCs
fig, ax = plt.subplots(figsize=(10, 8))
ax.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap="tab10", alpha=0.5, s=10)
```

5.2.2 t-SNE

```
from sklearn.manifold import TSNE

# For large datasets, first reduce to 50 dims with PCA
pca50 = PCA(n_components=50, random_state=42)
X_pca50 = pca50.fit_transform(X_scaled)

tsne = TSNE(
    n_components=2, perplexity=30,      # 5–50, larger=more global
    learning_rate="auto", n_iter=1000,
    metric="euclidean", random_state=42,
    n_jobs=-1
)
X_tsne = tsne.fit_transform(X_pca50)
```

5.2.3 UMAP

```
import umap

reducer = umap.UMAP(
    n_components=2, n_neighbors=15,     # local vs global structure
    min_dist=0.1,                       # how tightly packed embeddings are
    metric="euclidean", random_state=42
)
X_umap = reducer.fit_transform(X_scaled)
```

5.3 Anomaly Detection

```
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM

# Isolation Forest — fast, tree-based, scales well
iso = IsolationForest(
    contamination=0.05,     # expected fraction of anomalies
    n_estimators=200, max_features=1.0,
    random_state=42, n_jobs=-1
)
```

```
anomalies = iso.fit_predict(X)          # -1 = anomaly, 1 = normal
scores    = iso.score_samples(X)        # lower = more anomalous

# Local Outlier Factor — density-based
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05, n_jobs=-1)
lof_labels = lof.fit_predict(X)
```

© 2025 AI & ML Handbook

CHAPTER 6

Neural Networks

From Perceptrons to Deep Learning

6.1 The Artificial Neuron

An artificial neuron computes a weighted sum of its inputs, adds a bias, then applies a non-linear activation function: output = activation(w·x + b). Stacking layers of neurons yields a multilayer perceptron (MLP).

Why Depth Matters

Deeper networks learn hierarchical representations. A shallow network may require exponentially many neurons to represent what a deep network represents efficiently. The Universal Approximation Theorem guarantees that even a single hidden layer can approximate any continuous function — but depth makes learning tractable in practice.

6.2 Activation Functions

Function	Formula	Best Used For
ReLU	`max(0, x)`	Hidden layers — fast, sparse activations
Leaky ReLU	`max(0.01x, x)`	Avoid dying ReLU problem
GELU	`x·Φ(x)`	Transformers, BERT, GPT
Sigmoid	`1/(1+e^-x)`	Binary output layer
Softmax	$e^{x_i}/\Sigma e^{x_j}$	Multi-class output layer
Tanh	`(e^x-e^-x)/(e^x+e^-x)`	RNNs, bounded outputs
Swish	`x·sigmoid(x)`	EfficientNet, deep networks

6.3 PyTorch — Building Neural Networks

6.3.1 Tensors

```
import torch
import torch.nn as nn
import torch.nn.functional as F

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Tensor creation
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
z = torch.zeros(3, 4, device=device)
r = torch.randn(1000, 784, device=device)

# Operations
```

```
y = x @ x.T                              # matrix multiply
y = torch.sigmoid(x)
y = F.relu(x)

# NumPy interop
arr = x.cpu().numpy()
t   = torch.from_numpy(arr)
```

6.3.2 Defining a Model

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden, out_dim, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden),
            nn.BatchNorm1d(hidden),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, hidden // 2),
            nn.BatchNorm1d(hidden // 2),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden // 2, out_dim),
        )
    def forward(self, x):
        return self.net(x)

model = MLP(in_dim=784, hidden=512, out_dim=10).to(device)
print(model)
print(f"Parameters: {sum(p.numel() for p in model.parameters()):,}")
```

6.3.3 Training Loop

```
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Dataset and DataLoader
dataset = TensorDataset(X_tensor, y_tensor)
train_loader = DataLoader(dataset, batch_size=256, shuffle=True, num_workers=4)

# Loss and optimizer
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100)

# Training loop
for epoch in range(100):
    model.train()
    epoch_loss = 0.0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        logits = model(X_batch)
        loss   = criterion(logits, y_batch)
        loss.backward()
```

© 2025 AI & ML Handbook

```
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        epoch_loss += loss.item()
    scheduler.step()
    if (epoch+1) % 10 == 0:
        print(f"Epoch {epoch+1} | Loss: {epoch_loss/len(train_loader):.4f}")
```

6.3.4 Saving and Loading

```
# Save
torch.save({
    "epoch": epoch,
    "model_state_dict":  model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "loss": epoch_loss,
}, "checkpoint.pth")

# Load
ckpt = torch.load("checkpoint.pth", map_location=device)
model.load_state_dict(ckpt["model_state_dict"])
optimizer.load_state_dict(ckpt["optimizer_state_dict"])
```

CHAPTER 7

Convolutional Neural Networks

Computer Vision with PyTorch & Keras

7.1 CNN Architecture

Convolutional layers apply learned filters across the spatial dimensions of an input, sharing weights and exploiting translation invariance. The typical CNN architecture stacks: Conv → BatchNorm → Activation → Pooling blocks, followed by fully-connected classification heads.

7.2 Image Classification with PyTorch

7.2.1 Data Loading and Augmentation

```
import torchvision.transforms as T
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

train_transform = T.Compose([
    T.RandomResizedCrop(224, scale=(0.7, 1.0)),
    T.RandomHorizontalFlip(),
    T.RandomRotation(15),
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05),
    T.RandomGrayscale(p=0.1),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

val_transform = T.Compose([
    T.Resize(256), T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

train_ds = ImageFolder("data/train", transform=train_transform)
val_ds   = ImageFolder("data/val",   transform=val_transform)
train_dl    =    DataLoader(train_ds,    batch_size=64,    shuffle=True,    num_workers=8,
pin_memory=True)
```

7.2.2 Transfer Learning with ResNet

```
import torchvision.models as models

# Load pretrained model
backbone = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)

# Freeze backbone for feature extraction
for param in backbone.parameters():
    param.requires_grad = False

# Replace classification head
```

```
n_classes = len(train_ds.classes)
in_features = backbone.fc.in_features
backbone.fc = nn.Sequential(
    nn.Dropout(0.4),
    nn.Linear(in_features, 512),
    nn.ReLU(),
    nn.Linear(512, n_classes),
)
model = backbone.to(device)

# Fine-tune: unfreeze all layers with lower LR for backbone
optimizer = optim.AdamW([
    {"params": model.layer4.parameters(), "lr": 1e-4},
    {"params": model.fc.parameters(),     "lr": 1e-3},
], weight_decay=1e-4)
```

7.2.3 Popular Architectures

Model	Top-1 (IN)	Params	Best Use Case
ResNet-50	76.1%	25M	General baseline
EfficientNet-B4	83.0%	19M	Accuracy/size tradeoff
ConvNeXt-B	84.6%	89M	Modern CNN baseline
ViT-B/16	84.5%	86M	Large data, high accuracy
DINOv2 ViT-L	86.3%	307M	Self-supervised features

7.3 Object Detection

7.3.1 YOLOv8

```
from ultralytics import YOLO

# Load pretrained model
model = YOLO("yolov8n.pt")          # n=nano, s=small, m=medium, l=large

# Inference
results = model("image.jpg", conf=0.25, iou=0.45)
for r in results:
    boxes = r.boxes.xyxy.cpu().numpy()   # (x1,y1,x2,y2)
    confs = r.boxes.conf.cpu().numpy()
    clses = r.boxes.cls.cpu().numpy()
    r.save("output.jpg")

# Fine-tune on custom dataset
model.train(
    data="dataset.yaml",             # YOLO format dataset
    epochs=100, imgsz=640,
    batch=16, lr0=0.01,
    augment=True, mosaic=1.0,
    device=0,                        # GPU
)
```

7.4 Semantic Segmentation

```
import segmentation_models_pytorch as smp

model = smp.Unet(
    encoder_name="efficientnet-b4",
    encoder_weights="imagenet",
    in_channels=3,
    classes=21,                      # number of segmentation classes
    activation=None,                 # apply softmax in loss
)

loss_fn = smp.losses.DiceLoss(mode="multiclass")
```

© 2025 AI & ML Handbook

CHAPTER 8

Natural Language Processing

Transformers & Large Language Models

8.1 Text Preprocessing

```
import re, string
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", "", text)     # URLs
    text = re.sub(r"<[^>]+>", "", text)              # HTML tags
    text = re.sub(r"[^a-z0-9\s]", " ", text)       # punctuation
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens
              if t not in stop_words and len(t) > 2]
    return " ".join(tokens)
```

8.2 Word Embeddings

8.2.1 Word2Vec

```
from gensim.models import Word2Vec

sentences = [doc.split() for doc in corpus]
model = Word2Vec(
    sentences, vector_size=300,
    window=5,                    # context window
    min_count=5,                 # ignore rare words
    sg=1,                        # 1=skip-gram, 0=CBOW
    workers=8, epochs=10
)
vec = model.wv["machine"]                   # 300-d vector
similar = model.wv.most_similar("ai", topn=10)
```

8.3 The Transformer Architecture

The Transformer, introduced in "Attention Is All You Need" (Vaswani et al., 2017), relies entirely on self-attention mechanisms to model relationships between all tokens in a sequence, enabling full parallelization during training.

Self-Attention Formula

Attention(Q, K, V) = softmax(QK^T / $\sqrt{d_k}$) · V. Queries, keys, and values are linear projections of the input. The scaling by $\sqrt{d_k}$ prevents dot products from growing large in magnitude.

8.4 Hugging Face Transformers

8.4.1 Text Classification

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import TrainingArguments, Trainer
from datasets import load_dataset
import evaluate

# Load model and tokenizer
model_name = "distilbert-base-uncased"
tokenizer  = AutoTokenizer.from_pretrained(model_name)
model      = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=2
)

# Tokenize dataset
dataset = load_dataset("imdb")

def tokenize(batch):
    return tokenizer(batch["text"], truncation=True,
                     max_length=512, padding="max_length")

tokenized = dataset.map(tokenize, batched=True, batch_size=1000)

# Training
metric = evaluate.load("accuracy")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = logits.argmax(axis=-1)
    return metric.compute(predictions=predictions, references=labels)

args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    learning_rate=2e-5,
    warmup_ratio=0.1,
    weight_decay=0.01,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="accuracy",
    fp16=True,
)

trainer = Trainer(
```

```
    model=model, args=args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["test"],
    compute_metrics=compute_metrics,
)
trainer.train()
```

8.4.2 Named Entity Recognition

```
from transformers import pipeline

ner = pipeline("ner", model="dbmdz/bert-large-cased-finetuned-conll03-english",
               aggregation_strategy="simple", device=0)

result = ner("Apple was founded by Steve Jobs in Cupertino, California.")
# → [{"entity_group":"ORG","word":"Apple","score":0.99,...}, ...]
```

8.4.3 Question Answering

```
qa = pipeline("question-answering", model="deepset/roberta-base-squad2")

answer = qa({
    "question": "What year did BERT come out?",
    "context":  "BERT was introduced by Google in October 2018..."
})
print(answer["answer"])   # → "October 2018"
```

8.4.4 Text Generation with Causal LMs

```
from transformers import AutoModelForCausalLM, GenerationConfig

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-1B-Instruct",
    torch_dtype=torch.bfloat16,
    device_map="auto"
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.2-1B-Instruct")

inputs = tokenizer(prompt, return_tensors="pt").to(device)
gen_cfg = GenerationConfig(
    max_new_tokens=512,
    do_sample=True, temperature=0.7, top_p=0.9,
    repetition_penalty=1.1
)
output = model.generate(**inputs, generation_config=gen_cfg)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

8.5 Retrieval-Augmented Generation (RAG)

```
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains import RetrievalQA
from langchain.llms import HuggingFacePipeline
```

```
# 1. Chunk documents
splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=64)
chunks = splitter.split_documents(documents)

# 2. Embed and store
embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-large-en-v1.5")
vectordb = Chroma.from_documents(chunks, embeddings, persist_directory="./chroma")

# 3. Retrieval chain
retriever = vectordb.as_retriever(search_kwargs={"k": 5})
qa_chain  = RetrievalQA.from_chain_type(llm=llm, retriever=retriever,
                                        return_source_documents=True)
result = qa_chain.invoke({"query": "Explain self-attention"})
```

© 2025 AI & ML Handbook

CHAPTER 9

Recurrent Networks & Time Series

LSTMs, GRUs & Temporal Forecasting

9.1 Recurrent Neural Networks

RNNs process sequential data by maintaining a hidden state that carries information from previous time steps. Vanilla RNNs suffer from vanishing/exploding gradients for long sequences. LSTMs and GRUs address this with gating mechanisms.

9.2 LSTM — Long Short-Term Memory

```
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden, n_layers, n_classes, dropout):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(
            embed_dim, hidden, num_layers=n_layers,
            batch_first=True, dropout=dropout, bidirectional=True
        )
        self.attn  = nn.Linear(hidden * 2, 1)
        self.head  = nn.Linear(hidden * 2, n_classes)
        self.drop  = nn.Dropout(dropout)

    def forward(self, x):
        emb = self.drop(self.embedding(x))         # (B, L, E)
        out, _ = self.lstm(emb)                    # (B, L, 2H)
        # Attention pooling
        w = torch.softmax(self.attn(out), dim=1)   # (B, L, 1)
        ctx = (w * out).sum(dim=1)                 # (B, 2H)
        return self.head(self.drop(ctx))
```

9.3 Time Series Forecasting

9.3.1 LSTM Forecaster

```
class TimeSeriesLSTM(nn.Module):
    def __init__(self, n_features, hidden, n_layers, horizon, dropout):
        super().__init__()
        self.lstm  = nn.LSTM(n_features, hidden, n_layers,
                             batch_first=True, dropout=dropout)
        self.head  = nn.Linear(hidden, horizon)

    def forward(self, x):                          # x: (B, T, F)
        out, _ = self.lstm(x)
        return self.head(out[:, -1, :])            # last step → forecast
```

9.3.2 Classical Methods (statsmodels)

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.exponential_smoothing.ets import ETSModel
```

```
from prophet import Prophet

# SARIMA
sarima = SARIMAX(y_train, order=(2,1,2), seasonal_order=(1,1,1,12))
sarima_fit = sarima.fit(disp=False)
forecast = sarima_fit.forecast(steps=24)

# Prophet — handles seasonality and holidays automatically
df_prophet = pd.DataFrame({"ds": dates, "y": values})
m = Prophet(yearly_seasonality=True, weekly_seasonality=True,
            changepoint_prior_scale=0.05)
m.add_country_holidays(country_name="US")
m.fit(df_prophet)
future = m.make_future_dataframe(periods=90)
forecast = m.predict(future)
```

© 2025 AI & ML Handbook

CHAPTER 10

Reinforcement Learning

Agents, Environments & Policy Optimization

10.1 Core Concepts

Reinforcement Learning trains an agent to take actions in an environment to maximize cumulative reward. The agent observes state s_t, selects action a_t, receives reward r_t, and transitions to next state s_{t+1}.

Concept	Description
Policy π(a|s)	Probability distribution over actions given state
Value V(s)	Expected cumulative discounted reward from state s
Q-function Q(s,a)	Expected return from taking action a in state s, then following π
Advantage A(s,a)	A(s,a) = Q(s,a) − V(s): how much better is action a than average?
Discount γ	Tradeoff between immediate and future rewards (typically 0.99)

10.2 Deep Q-Networks (DQN)

```
import gymnasium as gym
from collections import deque, namedtuple
import random

Transition = namedtuple("Transition", ["state","action","reward","next_state","done"])

class ReplayBuffer:
    def __init__(self, capacity):
        self.buf = deque(maxlen=capacity)
    def push(self, *args):
        self.buf.append(Transition(*args))
    def sample(self, batch_size):
        return random.sample(self.buf, batch_size)
    def __len__(self):
        return len(self.buf)

class DQN(nn.Module):
    def __init__(self, n_obs, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_obs, 256), nn.ReLU(),
            nn.Linear(256, 256),   nn.ReLU(),
            nn.Linear(256, n_actions),
        )
    def forward(self, x):
        return self.net(x)
```

10.3 Proximal Policy Optimization (PPO)

```
# Using Stable-Baselines3
from stable_baselines3 import PPO, SAC, TD3
from stable_baselines3.common.env_util import make_vec_env
from stable_baselines3.common.callbacks import EvalCallback

env = make_vec_env("LunarLander-v2", n_envs=8)

model = PPO(
    "MlpPolicy", env,
    learning_rate=3e-4,
    n_steps=2048,           # rollout buffer size
    batch_size=64,
    n_epochs=10,            # optimization epochs per rollout
    gamma=0.99,
    gae_lambda=0.95,        # GAE parameter
    clip_range=0.2,         # PPO clipping
    ent_coef=0.01,          # entropy bonus
    vf_coef=0.5,
    max_grad_norm=0.5,
    verbose=1,
)

eval_cb = EvalCallback(eval_env, best_model_save_path="./best",
                       eval_freq=10000, n_eval_episodes=20)
model.learn(total_timesteps=2_000_000, callback=eval_cb)
```

CHAPTER 11

Generative AI

Autoencoders, GANs & Diffusion Models

11.1 Variational Autoencoders (VAE)

```
class VAE(nn.Module):
    def __init__(self, in_dim, hidden, latent):
        super().__init__()
        # Encoder
        self.enc = nn.Sequential(
            nn.Linear(in_dim, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden),  nn.ReLU(),
        )
        self.mu_layer  = nn.Linear(hidden, latent)
        self.log_var   = nn.Linear(hidden, latent)
        # Decoder
        self.dec = nn.Sequential(
            nn.Linear(latent, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden),  nn.ReLU(),
            nn.Linear(hidden, in_dim),  nn.Sigmoid(),
        )

    def reparameterize(self, mu, log_var):
        std = torch.exp(0.5 * log_var)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        h      = self.enc(x.view(x.size(0), -1))
        mu, lv = self.mu_layer(h), self.log_var(h)
        z      = self.reparameterize(mu, lv)
        recon  = self.dec(z)
        return recon, mu, lv

def vae_loss(recon, x, mu, log_var, beta=1.0):
    recon_loss = F.binary_cross_entropy(recon, x.view(x.size(0),-1), reduction="sum")
    kl_div     = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return recon_loss + beta * kl_div
```

11.2 Generative Adversarial Networks

```
class Generator(nn.Module):
    def __init__(self, z_dim, img_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 256),     nn.LeakyReLU(0.2),
            nn.Linear(256, 512),        nn.LeakyReLU(0.2),
            nn.Linear(512, 1024),       nn.LeakyReLU(0.2),
            nn.Linear(1024, img_dim),   nn.Tanh(),
```

```
        )
    def forward(self, z): return self.net(z)

class Discriminator(nn.Module):
    def __init__(self, img_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(img_dim, 1024),   nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(1024, 512),        nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(512, 256),         nn.LeakyReLU(0.2), nn.Dropout(0.3),
            nn.Linear(256, 1),           nn.Sigmoid(),
        )
    def forward(self, x): return self.net(x)
```

11.3 Diffusion Models with Diffusers

```
from diffusers import StableDiffusionPipeline, DPMSolverMultistepScheduler

pipe = StableDiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-2-1",
    torch_dtype=torch.float16
).to("cuda")
pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)

image = pipe(
    prompt="A photorealistic mountain lake at sunset, 8K, ultra-detailed",
    negative_prompt="blurry, low quality, cartoon",
    num_inference_steps=25,
    guidance_scale=7.5,
    width=768, height=512,
    generator=torch.Generator("cuda").manual_seed(42)
).images[0]
image.save("output.png")
```

CHAPTER 12

MLOps

Experiment Tracking, Deployment & Monitoring

12.1 Experiment Tracking with MLflow

```
import mlflow
import mlflow.sklearn
from mlflow.models.signature import infer_signature

mlflow.set_experiment("my-classification-experiment")

with mlflow.start_run(run_name="xgb-v3"):
    # Log parameters
    mlflow.log_params({
        "n_estimators": 500, "learning_rate": 0.05,
        "max_depth": 6, "subsample": 0.8,
    })

    # Train
    model.fit(X_train, y_train)

    # Log metrics
    mlflow.log_metrics({
        "train_auc": roc_auc_score(y_train, model.predict_proba(X_train)[:,1]),
        "test_auc":  roc_auc_score(y_test,  model.predict_proba(X_test)[:,1]),
    })

    # Log model with signature
    signature = infer_signature(X_train, model.predict(X_train))
    mlflow.sklearn.log_model(model, "model",
        registered_model_name="xgb-classifier",
        signature=signature)

    mlflow.log_artifact("confusion_matrix.png")
```

12.2 Model Serialization

```
import joblib, pickle

# Joblib — recommended for scikit-learn (compression, parallelism)
joblib.dump(pipeline, "model.joblib", compress=("gzip", 3))
loaded = joblib.load("model.joblib")

# PyTorch — ONNX export for cross-framework deployment
dummy = torch.randn(1, 3, 224, 224, device=device)
torch.onnx.export(
    model, dummy, "model.onnx",
```

```
    input_names=["image"], output_names=["logits"],
    dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
    opset_version=17,
)
```

12.3 Serving with FastAPI

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import uvicorn

app = FastAPI(title="ML Model API", version="1.0")
model = joblib.load("model.joblib")

class PredictRequest(BaseModel):
    features: list[list[float]]

class PredictResponse(BaseModel):
    predictions: list[int]
    probabilities: list[list[float]]

@app.post("/predict", response_model=PredictResponse)
async def predict(req: PredictRequest):
    try:
        X = np.array(req.features)
        preds = model.predict(X).tolist()
        probs = model.predict_proba(X).tolist()
        return PredictResponse(predictions=preds, probabilities=probs)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
async def health(): return {"status": "ok"}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000, workers=4)
```

12.4 Containerization with Docker

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

# Build and run
# docker build -t ml-api:1.0 .
```

© 2025 AI & ML Handbook

```
# docker run -p 8000:8000 --gpus all ml-api:1.0
```

12.5 Model Monitoring

Deployed models degrade over time due to data drift, concept drift, and infrastructure changes. Monitoring is essential for production ML.

Monitoring Checklist

Track: (1) Prediction distributions — alert when they shift significantly. (2) Input feature distributions — Kolmogorov-Smirnov or PSI tests. (3) Business metrics — conversion rates, revenue impact. (4) Latency and error rates. (5) Ground truth labels — retrain when performance degrades.

```
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, ClassificationPreset

report = Report(metrics=[DataDriftPreset(), ClassificationPreset()])
report.run(reference_data=X_train_df, current_data=X_prod_df,
           column_mapping=column_mapping)
report.save_html("drift_report.html")
```

CHAPTER 13

Ethics & Responsible AI

Fairness, Explainability & Privacy

13.1 Algorithmic Fairness

A model can be statistically accurate while exhibiting discriminatory behavior toward protected groups. Fairness requires careful definition and measurement.

Fairness Criterion	Definition	Appropriate When
Demographic Parity	P(ŷ=1|A=a) = P(ŷ=1|A=b) for all groups a,b	Representation matters most
Equal Opportunity	Equal true positive rates across groups	Benefits distributed fairly
Equalized Odds	Equal TPR and FPR across groups	Errors equalized
Calibration	P(y=1|ŷ=p,A=a) = p for all groups	Probability scores used

13.2 Model Explainability

13.2.1 SHAP — SHapley Additive exPlanations

```
import shap

# Tree-based models
explainer = shap.TreeExplainer(xgb_model)
shap_values = explainer.shap_values(X_test)

# Global feature importance
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

# Local explanation — why did the model predict this instance?
shap.waterfall_plot(explainer(X_test[0:1])[0])

# Force plot
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0])
```

13.2.2 LIME

```
from lime.lime_tabular import LimeTabularExplainer

lime_exp = LimeTabularExplainer(
    X_train, feature_names=feature_names,
    class_names=["No", "Yes"], discretize_continuous=True
)
explanation = lime_exp.explain_instance(
    X_test[0], model.predict_proba, num_features=10
)
explanation.show_in_notebook()
```

© 2025 AI & ML Handbook

13.3 Privacy-Preserving ML

13.3.1 Differential Privacy

```
from opacus import PrivacyEngine

privacy_engine = PrivacyEngine()
model, optimizer, train_loader = privacy_engine.make_private_with_epsilon(
    module=model,
    optimizer=optimizer,
    data_loader=train_loader,
    epochs=10,
    target_epsilon=8.0,         # privacy budget (lower=more private)
    target_delta=1e-5,
    max_grad_norm=1.0,          # gradient clipping
)
```

13.3.2 Federated Learning

```
import flwr as fl

class FlowerClient(fl.client.NumPyClient):
    def get_parameters(self, config):
        return [val.cpu().numpy() for val in model.state_dict().values()]

    def fit(self, parameters, config):
        set_params(model, parameters)
        train(model, train_loader, epochs=config["epochs"])
        return self.get_parameters({}), len(train_loader.dataset), {}

    def evaluate(self, parameters, config):
        set_params(model, parameters)
        loss, accuracy = evaluate(model, val_loader)
        return loss, len(val_loader.dataset), {"accuracy": accuracy}

fl.client.start_numpy_client(server_address="127.0.0.1:8080",
                             client=FlowerClient())
```

Appendix: Quick Reference

A. Choosing the Right Algorithm

Problem	Data Size	Recommended Algorithm
Binary classification	<10K rows	Logistic Regression, SVM, Random Forest
Binary classification	>10K rows	XGBoost / LightGBM / Stacking
Multi-class	Any	Random Forest → Gradient Boosting → MLP
Regression	<10K	Lasso / Ridge / SVR
Regression	>10K	XGBoost / LightGBM / Neural net
Image classification	Any	ResNet / EfficientNet (transfer learning)
Text classification	Any	DistilBERT / RoBERTa fine-tuning
Clustering	<100K	K-Means++ / DBSCAN / Agglomerative
Anomaly detection	Any	Isolation Forest / LOF / Autoencoder
Time series	Any	Prophet / SARIMA / LSTM / Transformer
Reinforcement Learning	N/A	PPO (continuous) / DQN (discrete)

B. Essential Library Cheatsheet

Library	Purpose	Key Classes / Functions
NumPy	Numerical arrays	`ndarray, broadcasting, linalg, random`
Pandas	Tabular data	`DataFrame, groupby, merge, pivot_table`
Scikit-learn	Classical ML	`Pipeline, ColumnTransformer, GridSearchCV`
PyTorch	Deep learning	`Module, DataLoader, optim, autograd`
TensorFlow/Keras	Deep learning	`Sequential, Model, callbacks, layers`
XGBoost	Gradient boosting	`XGBClassifier, XGBRegressor, DMatrix`
LightGBM	Fast GBDT	`LGBMClassifier, Dataset, early_stopping`
Transformers	Pre-trained LLMs	`AutoModel, pipeline, Trainer, PEFT`
OpenCV	Computer vision	`imread, resize, cvtColor, findContours`
SHAP	Explainability	`TreeExplainer, summary_plot, waterfall_plot`
MLflow	Experiment tracking	`start_run, log_params, log_metrics, log_model`

C. Common Loss Functions

© 2025 AI & ML Handbook

Loss	Task	PyTorch
MSE	Regression	`nn.MSELoss()`
MAE (L1)	Robust regression	`nn.L1Loss()`
Huber	Regression + outliers	`nn.HuberLoss(delta=1.0)`
BCE	Binary classification	`nn.BCEWithLogitsLoss()`
Cross-Entropy	Multi-class	`nn.CrossEntropyLoss(label_smoothing=0.1)`
Focal	Imbalanced classification	`torchvision.ops.sigmoid_focal_loss()`
KL Divergence	VAE, distillation	`nn.KLDivLoss(reduction="batchmean")`
Contrastive	Metric learning	`nn.CosineEmbeddingLoss()`

D. Optimization Algorithms

Optimizer	Characteristics	Typical LR
SGD+Momentum	Simple, needs LR tuning, best for CV with warmup	0.01 – 0.1
Adam	Adaptive LR, fast convergence, may generalize less well	1e-3
AdamW	Adam + decoupled weight decay — default for transformers	1e-4 – 3e-4
Lion	Memory efficient, sign-based updates	1e-4
Prodigy	Self-tuning LR, no LR search needed	1.0 (auto)

E. Recommended Learning Path

1. Master Python, NumPy, Pandas — weeks 1–4
2. Study probability, statistics, and linear algebra
3. Work through scikit-learn's user guide with real datasets (Kaggle)
4. Implement neural networks from scratch in NumPy, then PyTorch
5. Complete a CNN project on image classification
6. Fine-tune a Hugging Face model for NLP
7. Build an end-to-end ML project: data → model → API → monitoring
8. Study MLOps — MLflow, Docker, CI/CD for ML
9. Explore specializations: RL, generative models, or domain-specific ML
10. Read seminal papers: Attention Is All You Need, ResNet, BERT, GPT-3, AlphaFold

---

All code examples are compatible with Python 3.10+ and current library versions as of 2025.